

Guía de Estudio

Frontend **Ventas** — Angular 21

Bitácora razonada de cambios y glosario de conceptos

Proyecto **Ventas-Frontend** (**demo-ira-car**)

Documento generado a partir de la bitácora del 9 al 17 de septiembre de 2026

Uso: material de repaso / estudio del código

Índice

1. Contexto general del proyecto
2. Glosario de conceptos base
3. Bitácora razonada de cambios
 - 3.1 Imágenes de productos (2026-09-09)
 - 3.2 Fixes UI/UX + change detection + animaciones (2026-09-10)
 - 3.3 Seguridad + preparación Netlify/Railway (2026-09-12)
 - 3.4 Paginación client-side (2026-09-12)
 - 3.5 Filtros client-side (2026-09-12)
 - 3.6 Tests en verde + filtros Salehistory + bugs (2026-09-15)
 - 3.7 Botones Editar/Cancelar unificados (2026-09-15)
 - 3.8 Rediseño Compras/Reportes/Salehistory + limpieza de rutas (2026-09-17)
4. Patrones y convenciones del proyecto
5. Mapa de archivos
6. Comandos de trabajo

1. Contexto general del proyecto

1.1 Stack tecnológico

Capa	Tecnología	Para qué se usa
Framework	Angular 21 (standalone)	SPA sin <code>NgModules</code> ; cada componente declara sus propios imports.
Lenguaje	TypeScript ~5.9	Tipado estático de modelos e interfaces.
Reactividad	RxJS 7	<code>Observable</code> de HTTP, <code>BehaviorSubject</code> del carrito.
UI	Angular Material 21 + CDK	Date pickers, diálogos, snackbars.
Reportes	Chart.js 4, xlsx, jsPDF + autotable	Gráficas y exportaciones 100% en el navegador.
Change detection	zone.js (reactivado)	Que las respuestas HTTP repinten la UI automáticamente.
Testing	Vitest + jsdom	Pruebas unitarias de componentes y servicios.
SSR	Angular SSR	Prerender para despliegue (Netlify / nginx).

1.2 Arquitectura de carpetas

```
src/app/
├─ app.ts / app.html / app.css      → Root (solo <router-outlet>)
├─ app.config.ts                    → providers globales (router, http + interceptor)
├─ app.routes.ts                     → rutas (lazy load por componente)
├─ core/
│   ├── interfaces/                  → login, product, sale, payment, cobro, user, purchases, reports
│   ├── models/                      → role-model.ts
│   ├── enums/paymentMethod.ts       → CASH | DEBIT | CREDIT
│   ├── utils/storage-utils.ts        → safeGetItem/safeSetItem/safeRemoveItem (SSR-safe)
│   ├── pipes/paginate/              → PaginatePipe (paginación client-side)
│   ├── components/pagination-control/ → PaginationControl (pie de tabla reutilizable)
│   ├── service/                     → auth, product, sale, cash, payment, user, role,
│   │                               category, cobro, ticket, permission, sidebar,
│   │                               provider, purchase, report
│   └─ routes/
│       ├── guards/permission-guard.ts
│       ├── directives/has-permission-*.ts
│       └─ auth-interceptor/auth-interceptor.ts
└─ features/
    ├── login/ + forgot-password/ + reset-password/
    ├── charge/ → cobro (POS) + facade/ (sale-facade, cash-facade) ← NÚCLEO
    ├── sidebar/ → menú lateral reutilizable
    ├── products/ → CRUD productos con imagen
    ├── categories/, users/, roles/ → CRUD completos
    ├── profile/ → datos + cambio de contraseña + logout
    ├── shopping/ → Compras (3 tabs: compras / proveedores / márgenes)
    ├── reports/ → Reportes (Chart.js + export PDF/XLSX)
    └─ salehistory/ → Historial de ventas (paginado + filtros)
```

1.3 Backend conectado

- Backend hermano **Ventas-Backend** — Spring Boot en `http://localhost:8081/api`.
- `environment.ts` (dev): `api: 'http://localhost:8081/api'`.
- `environment-prod.ts`: placeholder `https://TU-APP.up.railway.app/api`.
- JWT: `AuthInterceptor` agrega `Authorization: Bearer <token>` a cada request.

Cómo se relacionan frontend y backend

El frontend no guarda reglas de negocio críticas: solo las presenta. El carrito del POS vive en memoria, pero la venta real, el stock y los pagos los resuelve el backend. El frontend pide datos por HTTP y los pinta; los permisos de UI salen del `localStorage` (por eso el backend DEBE validar el JWT por endpoint — ver sección 3.3).

2. Glosario de conceptos base

Conceptos que aparecen una y otra vez en el código. Léelos antes de la bitácora: ahí se aplican con ejemplos reales.

2.1 Componente standalone

Angular 21 no usa `NgModules`. Cada componente declara lo que necesita en el arreglo `imports` de su decorador `@Component`. Así no hay una "mochila" global de imports y el lazy loading es más limpio.

```
@Component({
  selector: 'app-salehistory',
  imports: [CommonModule, FormsModule, Sidebar, PaginatePipe, PaginationControl],
  templateUrl: './salehistory.html',
  styleUrls: ['./salehistory.css'],
})
export class Salehistory implements OnInit { /* ... */ }
```

Punto clave

Si un template falla con `NG0303: Can't bind to 'ngForOf'` o similar, casi siempre falta importar `CommonModule` (directivas `ngIf/ngFor`, pipes `date`, `number`) o `FormsModule` (`ngModel`).

2.2 Observable vs BehaviorSubject

Un `Observable` es un flujo de datos al que te *suscribes*. Un `BehaviorSubject` es un observable especial que **guarda el último valor** y lo entrega de inmediato a quien se suscribe. Eso lo hace ideal para estado compartido.

```
// Estado compartido del menú lateral (singleton providedIn: 'root')
private menuOpen$ = new BehaviorSubject<boolean>(true);
menuOpen = this.menuOpen$; // se expone como Observable de solo lectura
toggle(v: boolean) { this.menuOpen$.next(v); }
```

El carrito del POS (`CobroService`) también usa `BehaviorSubject<CobroItem[]>`: cada componente que se suscribe ve el carrito actual y todos los cambios siguientes.

2.3 subscribe y change detection

`HttpClient.get()` devuelve un observable frío: hay que `.subscribe()` para disparar el request. El callback `next` recibe la respuesta.

```
this.saleService.getSales().subscribe({
  next: (data) => { this.sales = data; this.loading = false; },
  error: () => { this.loading = false; }
});
```

Aquí está la razón de reactivar zone.js (sección 3.2)

En apps *zoneless*, asignar `this.sales = data` dentro del callback NO redibuja la tabla sola: Angular no sabe que "algo cambió". zone.js parchea `XMLHttpRequest` / `fetch` y avisa a Angular para que corra la detección de cambios al terminar cada request.

2.4 Pipes

Un pipe transforma un dato en el template. Los "puros" (por defecto) solo recalculan cuando cambia la referencia de entrada.

- `{{ total | number:'1.2-2' }}` — formato numérico: mínimo 1 entero, 2 decimales.
- `{{ fecha | date:'dd/MM/yyyy HH:mm' }}` — formato de fecha.
- `{{ lista | paginate: page : pageSize }}` — pipe propio que recorta la lista.

2.5 ng-container y ng-template

`<ng-container>` agrupa elementos sin generar un nodo en el DOM. Es la forma idiomática de repetir **varias filas** por cada elemento de un `*ngFor` sin envolverlas (lo que rompería la tabla HTML).

2.6 Data binding

Sintaxis	Dirección	Ejemplo
<code>[prop]="expr"</code>	Componente → DOM	<code>[disabled]="!valido"</code>
<code>(event)="fn()"</code>	DOM → Componente	<code>(click)="abrirModal()"</code>
<code>[(ngModel)]="campo"</code>	Bidireccional	inputs de formulario
<code>[class.activo]="cond"</code>	Activa una clase CSS	<code>[class.active]="tab === 'compras'"</code>
<code>[ngValue]</code>	Objeto real en <code><option></code>	ver sección 3.5

[value] vs [ngValue]

`[value]` siempre entrega un string. `[ngValue]` entrega el valor **real** (number, boolean, objeto). Si comparas con `===` en un getter, necesitas `[ngValue]`, o `"5" !== 5` te romperá el filtro.

2.7 Guard funcional (CanActivateFn)

Función que decide si se puede entrar a una ruta. En este proyecto valida `route.data.permission` contra los permisos del usuario en `localStorage`.

```
// app.routes.ts
{ path: 'compras', component: Compras, canActivate: [permissionGuard],
  data: { permission: 'VER_COMPRAS' } }
```

2.8 Directiva estructural *hasPermission

Ocultar/mostrar UI según permisos sin tocar los componentes. Uso: `*hasPermission="'CREAR_COMPRAS'"`. Es la contraparte de los guards a nivel de fragmento.

2.9 Interceptor HTTP funcional

Se ejecuta en cada request/response. Aquí agrega el token y maneja el 401 global.

```
// Agrega Authorization: Bearer <token> y, ante un 401, limpia sesión y va a '/'
intercept(req, next) {
  const token = safeGetItem('token');
  const authReq = token ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } }) : req;
  return next(authReq).pipe(catchError(err => { /* 401 → logout */ })));
}
```

2.10 SSR-safe: guard de window

En SSR/prerender no existe `localStorage` ni `window`. Los helpers envuelven el acceso para que el build de Netlify/nginx no explote.

```
export function safeGetItem(key: string): string | null {
  if (typeof window === 'undefined') return null;
  try { return window.localStorage.getItem(key); } catch { return null; }
}
```

2.11 CSS que se repite en el proyecto

Concepto	Dónde	Por qué
Flexbox (<code>display:flex</code>)	toolbars, botones, tarjetas	Alinear en una dimensión.
Grid (<code>display:grid</code>)	formularios de filtros, líneas del modal	Columnas alineadas sin calcular anchos.
<code>position: sticky</code>	<code>thead</code> de tablas	Encabezado fijo al hacer scroll.
<code>font-variant-numeric: tabular-nums</code>	columnas de dinero	Dígitos de ancho fijo → montos alineados.
<code>overflow: hidden</code> + <code>border-radius</code>	<code>.table-card</code>	Que las esquinas redondeen también el pie de paginación.
<code>::before</code>	barra de acento de tarjetas	Dibujar la franja de color sin HTML extra.
<code>transition</code> / <code>@keyframes</code>	sidebar, tablas	Animaciones suaves (colapso 800ms, <code>fade-in-up</code>).

3. Bitácora razonada de cambios

Cada bloque sigue la misma estructura: **problema** → **razonamiento** → **solución** → **código clave** → **concepto**.

3.1 Imágenes de productos (2026-09-09) ok

Problema

Las imágenes de productos no se veían porque el backend devolvía rutas relativas o `null` y el frontend las usaba tal cual.

Razonamiento

El origen de la imagen puede cambiar (localhost → Railway). Si la URL es relativa, hay que completarla contra `environment.api`; si es absoluta, respetarla. Así las imágenes sobreviven al cambio de dominio sin tocar el backend.

Solución

```
// productos.ts - imageUrl()
imageUrl(p): string {
  if (!p.img) return 'assets/placeholder.png';
  return /^https?:\/\//.test(p.img) ? p.img : `${environment.api}${p.img}`;
}
```

Concepto

`/^https?:\/\//.test(x)` es una expresión regular que comprueba si el string empieza por `http://` o `https://`.

3.2 Fixes UI/UX + change detection + animaciones (2026-09-10) ok

3.2.1 SidebarService (estado compartido del menú)

- **Problema:** cada componente tenía su `menuOpen` local; al navegar el menú no se sincronizaba.
- **Solución:** un singleton (`providedIn: 'root'`) con `menuOpen$` y suscripción a `router.events`.
- **Detalle fino:** se escucha `NavigationStart` (no `NavigationEnd`) para que el cierre empiece al hacer clic y la animación de 800ms alcance a terminar antes de cargar el componente nuevo.

```
this.router.events
  .pipe(filter(e => e instanceof NavigationStart))
  .subscribe(() => this.toggle(false));
```

3.2.2 Reactivación de zone.js (el bug más importante)

Síntoma

Las tablas y el estado de caja no aparecían hasta que el usuario hacía clic en algún lado. La petición HTTP sí llegaba, pero la vista no se actualizaba.

Causa raíz: Angular 21 usa *zoneless* por defecto. Sin *zone.js*, los callbacks de `HttpClient.subscribe()` no disparan detección de cambios.

Fix (3 pasos):

```
npm install zone.js @angular/ssr@21.2.1

// angular.json
"polyfills": ["zone.js"]

// app.config.ts
providers: [ provideZoneChangeDetection(), /* ... */ ]
```

Concepto: change detection

Angular debe saber cuándo revisar los bindings. Zoneless depende de signals o de `markForCheck()` ; con *zone.js*, Angular intercepta eventos y *async* y revisa solo. Se reactivó porque el proyecto usa RxJS + **ngFor* clásicos.

3.2.3 Animaciones suaves

```
/* src/styles.css – colapso natural del sidebar */
.layout .sidebar { transition: width 800ms ease-in-out, padding 800ms ease-in-out; }

/* Tablas CRUD */
@keyframes fade-in-up {
  from { opacity: 0; transform: translateY(12px); }
  to   { opacity: 1; transform: translateY(0); }
}
.fade-in { animation: fade-in-up 250ms ease-out; }
```

Se aplica `class="table-container fade-in"` dentro de `<div *ngIf="!loading">` : la animación se reproduce justo al desaparecer el estado de carga.

3.3 Seguridad + preparación de despliegue (2026-09-12) ok

3.3.1 storage-utils.ts (helpers SSR-safe)

Centraliza el acceso a `localStorage` y evita errores en SSR/prerender y ante un `localStorage` manipulado.

```
export function safeGetItem(key: string): string | null {
  if (typeof window === 'undefined') return null; // SSR/prerender
  try { return window.localStorage.getItem(key); }
  catch { return null; } // contexto restringido / JSON roto
}
```

3.3.2 isLogged() valida la expiración del JWT

Antes bastaba con que existiera un token; si estaba expirado la app se veía "logueada" pero todo fallaba. Ahora se decodifica el payload (claim `exp`) y se rechaza el token vencido.

Concepto: JWT

Un JWT son 3 partes separadas por punto: `header.payload.signature`. El payload va en Base64 y contiene claims como `exp` (expiración, en segundos Unix). El frontend puede LEERLO para saber si expiró, pero **no puede validar la firma**: eso es del backend.

3.3.3 AuthInterceptor: 401 global

Si el backend responde 401, el interceptor limpia la sesión y redirige a `/`. Excluye `/auth/login`, `forgot-password` y `reset-password`, porque ahí un 401 es un fallo de credenciales normal, no una sesión vencida.

3.3.4 Despliegue Netlify + Railway

Archivo	Función
<code>angular.json</code>	<code>fileReplacements</code> en production → usa <code>environment-prod.ts</code> (antes era código muerto y todo apuntaba a localhost).
<code>public/_redirects</code>	<code>/* /index.html 200</code> → SPA fallback. Sin esto, recargar <code>/productos</code> da 404 en Netlify.
<code>Dockerfile</code>	<code>node:22-alpine</code> → build → copia <code>dist/.../browser</code> a nginx.
<code>nginx.conf</code>	SPA fallback + cache 1 año para assets con hash + headers de seguridad.

Pendiente de backend

El frontend lee `permissions` de `localStorage` (manipulable). El backend DEBE validar el JWT por endpoint. También debe permitir el origen de Netlify en CORS.

3.4 Paginación client-side (2026-09-12) ok

Razonamiento

Los servicios ya cargan TODOS los registros. En vez de tocar el backend, se pagina en memoria: más simple, instantáneo, sin endpoints nuevos. El precio es que no escala a miles de filas (ahí sí habría que pagar en servidor).

Solución

```
// core/pipes/paginate/paginate.ts (pipe Puro, standalone)
transform(list: T[], page: number, pageSize: number): T[] {
  const start = page * pageSize;
  return list.slice(start, start + pageSize); // clampa si la página quedó fuera de rango
}

// template
<tr *ngFor="let p of filteredProducts | paginate: page : pageSize"> ... </tr>
<app-pagination [total]="filteredProducts.length" [pageSize]="pageSize"
  [page]="page" (pageChange)="page = $event"></app-pagination>
```

¿Por qué el pipe clampa?

Si borras el último ítem de una página, **page** puede quedar fuera de rango y la tabla se vería vacía. El pipe ajusta la página al último índice válido.

Decisión: el carrito del POS NO se pagina — es un carrito vivo, no una tabla de registros.

3.5 Filtros client-side (2026-09-12) ok

Patrón

Estado de filtro en el componente + un **getter** que devuelve la lista filtrada. El ***ngFor** y el **[total]** de la paginación usan la lista filtrada, así el pie cuenta solo lo visible.

```
// componentes CRUD
selectedCategory: number | null = null; // null = "Todas"

get filteredProducts(): Product[] {
  if (this.selectedCategory === null) return this.products;
  return this.products.filter(p => p.categoryId === this.selectedCategory);
}

// al cambiar el criterio, volver a la primera página
onCategoryFilterChange() { this.page = 0; }
```

Trampa resuelta con [ngValue]

<option [value]="p.id"> entrega el string **"3"**, pero el getter compara **p.categoryId === 3** (número). Con **[ngValue]="p.id"** el select entrega un number real y la comparación estricta funciona. Igual con booleanos (Activo/Inactivo) en Usuarios.

La barra usa clases compartidas en `styles.css` : `.table-filter-bar` , `.table-filter-field` , `.filter-select` , `.filter-input` . Va como primer hijo de `.table-card` , con el mismo lenguaje visual que el pie de paginación.

3.6 Tests en verde + filtros Salehistory + bugs (2026-09-15) ok

3.6.1 Arreglo de 4 tests (de 15/19 a 19/19)

Spec	Fallo	Fix
<code>app.spec.ts</code>	Buscaba <code>Hello, Demo-IraCar</code> (plantilla de <code>ng new</code>)	Test "should render the router outlet" + <code>provideRouter([])</code>
<code>login.spec.ts</code>	<code>NG0201: No provider for ActivatedRoute</code>	<code>providers: [provideRouter([])]</code>
<code>forgot-password.spec.ts</code>	idem (usa <code>RouterLink</code>)	<code>provideRouter([])</code>
<code>reset-password.spec.ts</code>	idem (inyecta <code>ActivatedRoute</code>)	<code>provideRouter([])</code>

Concepto

Un `TestBed` se configura como una mini-app: si el componente en su template usa `routerLink` o inyecta `ActivatedRoute`, hay que ofrecer los providers del router o el test explota antes de correr.

3.6.2 Filtros client-side en Salehistory (fechas + método)

```
fromDate = '';
toDate = '';
selectedMethod: PaymentMethod | null = null;

get filteredSales(): SaleHistory[] {
  let result = this.sales;
  if (this.selectedMethod !== null) {
    result = result.filter(s => s.paymentMethod === this.selectedMethod);
  }
  if (this.fromDate || this.toDate) {
    result = result.filter(s => {
      const datePart = s.saleDate.substring(0, 10); // 'yyyy-MM-dd'
      if (this.fromDate && datePart < this.fromDate) return false;
      if (this.toDate && datePart > this.toDate) return false;
      return true;
    });
  }
  return result;
}
```

Truco: comparar fechas como strings

`saleDate` viene como ISO (`2026-09-15T10:30:00`). Al cortar los primeros 10 caracteres queda `2026-09-15` . Como ese formato es `yyyy-MM-dd` , comparar con `< / >` es equivalente a comparar cronológicamente y es inmune a timezones (a diferencia de `new Date()` , que puede correr el día).

3.6.3 Bugs menores

- Typo `Cambio` → `Cambio` en `sale-facade.ts:477`.
- `forgot-password.css` : `width: 350;` (sin unidad, CSS lo ignora) → `350px`.
- `roles.ts` : definía `ngOnInit` sin declarar `implements OnInit`.
- `perfil.ts` : leía `localStorage.getItem('user')` directo → ahora usa `auth.getUser()` (JSON corrupto devuelve null en vez de romper).

3.7 Botones Editar/Cancelar unificados (2026-09-15) ok

Problema

Cada CRUD tenía su propio estilo de botón "Editar": productos/usuarios usaban amarillo; categorías usaba `class="editRole"` (clase que ni existía → botón sin estilo); roles usaba `.editRole` azul sólido. Además solo categorías mostraba botón Cancelar al editar.

Razonamiento

Un lenguaje visual consistente reduce carga cognitiva: el usuario aprende una vez que amarillo = editar, rojo = eliminar, gris = cancelar. Se eligió `.cancel` gris (no rojo) para no confundir cancelar-edición con eliminar-registro.

Solución

- Categorías y roles pasan a `class="edit"`.
- Se agrega Cancelar en edición a productos, usuarios y roles: `<button *ngIf="editingXId" class="cancel" (click)="cancelEdit()">Cancelar</button>`. El `*ngIf` hace que solo aparezca en modo edición.
- Nueva clase `.cancel` (gris `#64748b` → `#475569`) en los 4 CSS.

```
/* Lenguaje de botones de acción, igual en los 4 CRUDs */
.edit { background: linear-gradient(135deg, #facc15, #eab308); } /* amarillo */
.delete { background: linear-gradient(135deg, #ef4444, #dc2626); } /* rojo */
.cancel { background: linear-gradient(135deg, #64748b, #475569); } /* gris */
```

```
// productos.ts – estado de edición + cancelar
editingProductId: number | null = null;

editProduct(p: ProductForm) { this.editingProductId = p.id ?? null; /* ... */ }
resetForm() { this.editingProductId = null; /* ... */ }
cancelEdit() { this.resetForm(); }
```

3.8 Rediseño Compras/Reportes/Salehistory + limpieza (2026-09-17) ok

Objetivo: alinear los 3 últimos módulos al lenguaje visual de los CRUDs (tarjetas `#1e293b` con borde `#263449`, botones `.edit/.delete/.cancel`, inputs sólidos `#0f172a`, montos `| number: '1.2-2'`).

3.8.1 Limpieza: placeholders eliminados

El build estaba roto

Los archivos de Caja, Ventas, Clientes y Facturas no existían en disco, pero `app.routes.ts` aún los importaba → `TS2307: Cannot find module`. Se quitaron las 4 rutas (`/clientes`, `/facturas`, `/ventas`, `/caja`). El sidebar ya no las enlazaba. Los specs asociados también se borraron (18/19 → 14/15).

Lección

Al borrar un componente con lazy loading, hay que quitar su ruta e `import()` en `app.routes.ts`, no solo el archivo. El compilador de rutas carga en build.

3.8.2 Salehistory: badges + resumen + montos

De texto plano a *badges* con color por método de pago, y un resumen analítico de la lista filtrada.

```
<span class="method-badge"
  [class.cash]="sale.paymentMethod === PaymentMethod.CASH"
  [class.debit]="sale.paymentMethod === PaymentMethod.DEBIT"
  [class.credit]="sale.paymentMethod === PaymentMethod.CREDIT">
  {{ formatPaymentMethod(sale.paymentMethod) }}
</span>
```

```
// salehistory.ts – resumen reactivo de lo filtrado
get filteredCount(): number { return this.filteredSales.length; }
get filteredTotal(): number {
  return this.filteredSales.reduce((sum, s) => sum + (s.total ?? 0), 0);
}
```

Getters como "computed" sin signals

Un getter se recalcula cada ciclo de detección: simple pero no gratis. Para listas pequeñas (historial paginado) es perfecto. Con listas grandes convendría memoizar o usar `computed()` de signals.

```
/* .money: derecha + dígitos tabulares; .total: amarillo semibold */
.money { text-align: right; font-variant-numeric: tabular-nums; }
.money.total { color: #facc15; font-weight: 600; }
```

`[class.cash]` es la forma de activar una clase CSS sin `*ngIf`: más limpio y con menos ciclos que envolver en condicionales.

3.8.3 Compras: tabs, detalle en fila completa, botones y estados vacíos

- **Tabs:** de botones sueltos a control segmentado (contenedor `#1e293b`, `.tab.active` azul `#2563eb` + sombra).
- **Botones:** `.editRole` → `.edit`; los "Cancelar" pasan de `.delete` (rojo) a `.cancel` (gris).
- **Detalle expandible:** antes era un `<div>` dentro de la celda de acciones (columna angosta). Ahora es una **fila completa** con `colspan="5"`.
- **Estados vacíos** en las 3 tablas + prop `loading`.
- **Montos** con `| number: '1.2-2'`; botón primario verde → azul.

```
<!-- ng-container agrupa las dos <tr> por compra sin inyectar nodos al DOM -->
<ng-container *ngFor="let c of filteredPurchases | paginate: page : pageSize">
  <tr>
    <td>{{ c.purchaseDate | date:'dd/MM/yyyy HH:mm' }}</td>
    <td class="money">{{ c.total | number:'1.2-2' }}</td>
    <td><button class="view" (click)="toggleDetalle(c.id)">Detalle</button></td>
  </tr>

  <!-- Fila de detalle: ocupa toda la tabla con colspan -->
  <tr *ngIf="detalleAbiertaIds.has(c.id)" class="detalle-row">
    <td colspan="5">
      <div class="detalle fade-in"> ... subtabla ... </div>
    </td>
  </tr>
</ng-container>
```

Por qué `<ng-container>` y no un `<div>`

Dentro de `<tbody>` solo pueden ir `<tr>`. Un `<div>` rompería el HTML. `<ng-container>` permite repetir dos filas hermanas lógicamente agrupadas sin generar un elemento real.

El estado de expansión se guarda en un `Set<number>`:

```
detalleAbiertaIds = new Set<number>();
detalleAbiertaIds.has(id) // ¿está abierto?
toggleDetalle(id: number) { /* agrega/quita del Set */ }
```

Estado vacío con mensaje diferenciado — clave para debugging de filtros:

```
<tr *ngIf="!loading && filteredPurchases.length === 0">
  <td colspan="5" class="empty-row">
    {{ purchases.length === 0
      ? 'No hay compras registradas'
      : 'No hay compras que coincidan con el filtro' }}
  </td>
</tr>
```

Lección de UX

Distinguir "no hay datos" de "los filtros no arrojan nada" evita que el usuario crea que se borraron los registros. El mismo patrón se usó en Salehistory.

3.8.4 Reportes: filtros sólidos, tarjetas con acento y corte de caja

- **Filtros:** de `rgba(255,255,255,0.06)` translúcido a sólido `#0f172a` + borde `#334155` , focus azul. Se invierte el icono del date picker con `::-webkit-calendar-picker-indicator` para que se vea sobre fondo oscuro.
- **Tarjetas resumen:** barra de acento superior con `::before` por métrica (`.accent-blue` Ventas, `.accent-green` Total vendido, `.accent-orange` Ticket promedio), montos con `$` y subtítulos descriptivos.
- **Gráficas:** cada header lleva un `.chart-dot` con el color de la serie en Chart.js → leyenda visual sin mirar la gráfica.
- **Corte de caja:** montos con `$` y `tabular-nums` ; `.card` alineada a `.table-card` .

```
/* Barra de acento superior: pseudo-elemento, sin HTML extra */
.card { position: relative; overflow: hidden; }
.card::before {
  content: ''; position: absolute; top: 0; left: 0; right: 0; height: 3px;
  background: var(--accent, #2563eb);
}
.card.accent-blue { --accent: #3b82f6; }
.card.accent-green { --accent: #22c55e; }
.card.accent-orange { --accent: #f97316; }
```

Custom properties (variables CSS)

Se define `--accent` en la clase de cada tarjeta y el `::before` la usa. Es DRY: la regla de dibujo se escribe una vez y cada tarjeta solo cambia el color.

3.8.5 Verificación de la tanda

Comando	Resultado
<code>ng build</code> (production)	OK — solo warning pre-existente <code>cobro.css</code> 9.33 kB > 8 kB
<code>npx ng test --watch=false</code>	14 archivos / 15 tests — todos pasan

4. Patrones y convenciones del proyecto

4.1 Estructura estándar de una tabla

```
<div class="table-container fade-in">
  <div class="table-card">
    <!-- 1) barra de filtros (primer hijo, opcional) -->
    <div class="table-filter-bar"> ... </div>

    <!-- 2) scroll + tabla -->
    <div class="table-scroll">
      <table>
        <thead><tr><th>...</th></tr></thead>
        <tbody> ... filas + estado vacío ... </tbody>
      </table>
    </div>

    <!-- 3) pie de paginación (último hijo) -->
    <app-pagination [total]="listaFiltrada.length" [pageSize]="pageSize"
      [page]="page" (pageChange)="page = $event"></app-pagination>
  </div>
</div>
```

El orden importa: `.table-card` tiene `overflow:hidden`, así el pie de paginación queda redondeado en las esquinas inferiores.

4.2 Semáforo de botones

Clase	Color	Significado
<code>.primary</code>	Azul <code>#2563eb</code>	Acción principal (crear, guardar)
<code>.edit</code>	Amarillo	Editar
<code>.delete</code>	Rojo	Eliminar (destructivo)
<code>.cancel</code>	Gris	Cancelar (neutro, NO destructivo)
<code>.view</code>	Neutro	Ver detalle

4.3 Filtros client-side (receta)

1. Prop de estado con `null` = "Todos".
2. Getter que devuelve la lista filtrada.
3. `*ngFor` y `[total]` usan la lista filtrada.
4. Opciones con `[ngValue]` (no `[value]`).
5. `(ngModelChange)` resetea `page = 0`.

6. Estado vacío diferenciado con/sin filtro.

4.4 Formato de dinero

En tablas se usa `| number: '1.2-2'`; el `$` se agrega literal cuando aplica (`{{ total | number: '1.2-2' }}`). Las columnas llevan `.money` (derecha + tabular-nums).

4.5 Manejo de errores HTTP

```
this.svc.accion().subscribe({
  next: (data) => { /* asignar */ },
  error: (err) => alert(err.error?.message || 'Mensaje genérico')
});
```

Pendiente conocido: migrar `alert` / `confirm` nativos a MatSnackBar/MatDialog para consistencia.

5. Mapa de archivos

5.1 Servicios HTTP (core/service)

Servicio	Base	Uso
AuthService	/auth	login, forgot/reset/change-password, me/authorities; token y usuario en localStorage
ProductService	/products	GET (con ?category=), POST/PUT multipart, DELETE, barcode, search
SaleService	/sales + /products	processSale, catálogo POS, barcode, historial
PaymentService	/payments	status/{tx} (polling), retry, reverse
CashService	/cash	active, open, close, summary
UserService	/users + /roles	CRUD usuarios, activar/desactivar, combo roles
RoleService	/roles	CRUD roles
PermissionService	/permissions	lista de permisos agrupados por módulo
CategoryService	/categories	CRUD (incluye updateCategory)
ProviderService	/providers	CRUD proveedores
PurchaseService	/purchases	listar / byProvider / crear / cancelar
ReportService	/reports	trend, top-products, payment-methods, categories, low-stock, summary
CobroService	—	carrito local en memoria (sin HTTP)
TicketService	—	generación de PDF (sin HTTP)

5.2 Archivos tocados en las últimas tandas

Archivo	Cambio
src/app/app.routes.ts	Se quitaron rutas huérfanas /clientes , /facturas , /ventas , /caja .
features/shopping/compras.*	Tabs segmentados, detalle en fila completa, botones unificados, loading , estados vacíos, montos formateados.
features/reports/reportes.*	Filtros sólidos, tarjetas con acento, chart-dots, corte de caja con \$.
features/salehistory/salehistory.*	Badges de método de pago, .money , resumen de lista filtrada.
features/categories/categorias.*	Botón editar .edit ; cancelar .cancel .
features/roles/roles.*	Editar amarillo; cancelar con editingRoleId .
features/products/productos.*	editingProductId + cancelEdit() .
features/users/usuarios.*	editingUserId + cancelEdit() .
styles.css	Clases compartidas: tablas, filtros, botones, animaciones.
core/utils/storage-utils.ts	Helpers SSR-safe.
core/pipes/paginate/ , core/components/pagination-control/	Paginación client-side reutilizable.
AGENTS.md	Bitácora y contexto del proyecto.

5.3 Rutas activas

/ (login) · /cobro · /perfil · /compras · /productos · /reportes · /usuarios · /salehistory · /categorias · /roles

6. Comandos de trabajo

```
npm install           # dependencias
npm start             # ng serve → http://localhost:4200
npm run build         # build producción (usa environment-prod.ts)
npx ng test --watch=false  # tests (Vitest) una sola vez
```

Si un test falla con TS2307

Faltan dependencias en node_modules (p. ej. chart.js o xlsx tras clonar). Basta con npm install .